ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

ENSAM Casablanca

Artificial Intelligence & Computer Science (IAGI)

Stabilising Imagination-Based Deep Reinforcement Learning via Symlog Loss Transformations

Rabyâ RAGHIB & Hiba HANI

Supervisor: Badr HIRCHOUA

A report submitted in partial fulfilment of the requirements of
ENSAM Casablanca
Artificial Intelligence and Computer Science (IAGI)

June 9, 2026

Declaration

We, Rabyâ RAGHIB and Hiba HANI, of the Department of Computer Science, Artificial Intelligence & Computer Science (IAGI) branch, ENSAM Casablanca, confirm that this is our own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. We understand that if failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

We give consent to a copy of our report being shared with future students as an exemplar.

We give consent for our work to be made available more widely to members of ENSAM Casablanca and the public with interest in teaching, learning and research.

Rabyâ RAGHIB & Hiba HANI
June 9, 2026

Abstract

Deep reinforcement learning agents that learn through imagined trajectories within a learned world model improve sample efficiency over purely model-free methods. However, these imagination-based approaches are sensitive to reward scale: when value predictions are trained with standard mean squared error losses, critic predictions can diverge when imagined returns span several orders of magnitude. This report investigates whether the symlog transformation, a symmetric logarithmic function recently introduced in DreamerV3, can stabilize this training.

We implement a simplified Dreamer-like architecture with a learned transition model and an actor-critic policy trained on imagined rollouts. We compare three configurations: a model-free actor-critic baseline, an imagination-based actor-critic with standard MSE losses, and an imagination-based actor-critic with symlog-transformed losses. We evaluate these configurations on CartPole-v1 and LunarLander-v3 using five random seeds, yielding thirty training runs.

Without the symlog transformation, imagination-based training suffers from critic loss divergence (reaching values near 10^{15} in LunarLander) and performance collapse. The symlog transformation prevents this divergence, maintaining stable critic losses and performance. All configurations converge on the simpler CartPole-v1 environment, which confirms that the instability depends on reward scale.

These findings indicate that symlog-transformed losses are important for stable imagination-based training, highlighting the role of loss design in reinforcement learning.

Keywords: deep reinforcement learning, symlog, world model, imagination-based training, actor-critic, loss function adaptation

Acknowledgements

We express our sincere gratitude to our supervisor and Deep Reinforcement Learning course instructor, Mr. Badr HIRCHOUA, at ENSAM Casablanca for the guidance that made this project possible. The theoretical foundations and practical insights from his course helped shape our understanding of this field.

We are grateful to each other, Rabyâ and Hiba, for the collaboration and mutual support that carried this project from conception to completion. This work is the result of a team effort.

Finally, we acknowledge the open-source community whose tools enabled our experiments. In particular, we thank the developers of PyTorch and Gymnasium for providing the frameworks that allowed us to focus on the core research questions.

Contents

List of Figures

List of Tables

List of Abbreviations

AC	Actor-Critic
DRL	Deep Reinforcement Learning
ENSAM	École Nationale Supérieure d'Arts et Métiers
IAGI	Intelligence Artificielle et Génie Informatique
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
RL	Reinforcement Learning
RSSM	Recurrent State-Space Model
TD	Temporal Difference

General Introduction

Deep reinforcement learning (DRL) has been applied to sequential decision-making tasks, including games and robotic control. However, standard loss functions remain sensitive to the scale of rewards encountered during training. When reward magnitudes vary across environments or change dynamically, regression losses such as mean squared error (MSE) can produce large gradient updates, destabilizing the training process and causing value function divergence.

This challenge is significant in *imagination-based* reinforcement learning. Model-based approaches, such as the Dreamer family of agents, can improve sample efficiency by allowing training on simulated experience. However, because imagined rollouts are generated autoregressively, small prediction errors compound across time steps. Combined with scale-sensitive losses, this compounding effect can cause predicted values and rewards to diverge, leading to a value explosion.

DreamerV3 addressed this problem by introducing *symlog*-transformed losses. The symlog function, defined as $\text{symlog}(x) = \text{sign}(x) \cdot \ln(|x| + 1)$, compresses large magnitudes logarithmically while preserving the sign. By applying this transformation to critic and reward prediction targets before computing the MSE loss, DreamerV3 achieved scale-invariant learning without requiring task-specific hyperparameter tuning.

This project investigates whether this symlog transformation retains its stabilizing properties when integrated into a simplified imagination-based agent. Specifically, we implement a simplified Dreamer-like agent in PyTorch that operates in the raw state space rather than a learned latent space. The agent uses a Dyna-style training scheme, learning from both real environment interactions and imagined trajectories generated by a learned world model. The actor, critic, and world model components are implemented as two-layer multilayer perceptrons with 256 hidden units each.

We compare three configurations:

- **Condition A (Model-Free Actor-Critic):** A baseline agent that learns exclusively from real environment transitions.
- **Condition B (Imagination AC with Standard MSE):** An imagination-augmented agent that trains on both real and imagined data, using standard MSE losses.
- **Condition C (Imagination AC with Symlog MSE):** An imagination-augmented agent using symlog-transformed targets in the critic and reward prediction losses.

We evaluate these configurations on CartPole-v1 and LunarLander-v3 using five random seeds, yielding 30 runs.

The results reveal distinct differences between the configurations. Without symlog transformations, imagination-augmented training leads to value divergence: critic losses exceed 10^{15}, causing policy collapse. Symlog-transformed losses prevent this divergence, maintaining stable losses throughout training. These findings indicate that symlog is a broadly applicable technique for stabilizing imagination-based reinforcement learning.

Organisation of the Report

This report contains seven chapters. Chapter 1 introduces the background, problem statement, objectives, and solution approach. Chapter 2 reviews literature on model-free and model-based reinforcement learning, the Dreamer family, and the symlog transformation. Chapter 3 details the methodology, including agent architectures, training procedures, and environments. Chapter 4 presents the experimental results. Chapter 5 discusses the findings, study limitations, and broader context. Chapter 6 summarizes the conclusions and proposes future work. Finally, Chapter 7 provides a reflection on the project.

Chapter 1

Introduction

This chapter provides the background for the project. It introduces deep reinforcement learning, the challenges of reward scale sensitivity, our experimental configurations, and the structure of the report.

1.1 Background

Reinforcement learning (RL) is a machine learning framework in which an agent learns to make sequential decisions by interacting with an environment to maximize a cumulative reward signal. Over the past decade, combining reinforcement learning with deep neural networks, known as deep reinforcement learning (DRL), has enabled performance in board games, video games, and robotics.

DRL methods can be categorized into model-free and model-based approaches. Model-free methods, such as actor-critic algorithms, learn a policy or value function directly from sampled transitions. Although widely applicable, model-free methods are often sample-inefficient, requiring millions of interactions because transitions are used only a few times for updates.

Model-based reinforcement learning (MBRL) addresses this limitation by learning a world model that predicts environment transitions and rewards. Once a world model is trained, the agent can generate synthetic experience by simulating future trajectories, augmenting its training data. This imagination-based approach can improve sample efficiency, as real transitions train a world model that generates many simulated transitions for policy optimization.

The Dreamer family of agents is widely used in imagination-based reinforcement learning. The original Dreamer agent trained actor and critic networks within imagined latent trajectories generated by a recurrent state-space model. DreamerV2 introduced discrete latent representations to improve performance on Atari benchmarks. DreamerV3 introduced innovations to enable a single set of hyperparameters to work across environments with different reward scales, observation modalities, and action spaces.

Among these innovations, the symlog transformation addresses reward scale sensitivity. The symlog function is defined as:

$$\text{symlog}(x) = \text{sign}(x) \cdot \ln(|x| + 1), \tag{1.1}$$

with its inverse given by:

$$\text{symexp}(x) = \text{sign}(x) \cdot \left(e^{|x|} - 1\right). \tag{1.2}$$

By applying this transformation to the targets of the critic and reward prediction losses before computing the mean squared error, DreamerV3 compresses large magnitudes logarithmically

while preserving the sign, reducing sensitivity to reward scale. DreamerV3 used this approach to achieve high performance across benchmarks without environment-specific tuning.

1.2 Problem Statement

Standard mean squared error (MSE) losses are sensitive to the scale of target values. Large targets can produce large gradients, leading to unstable updates. In supervised learning, this is addressed via data normalization. In reinforcement learning, however, value estimates and predicted rewards are non-stationary signals that change during training.

This sensitivity to reward scale is a challenge for imagination-based agents. During autoregressive rollouts, prediction errors in rewards or state transitions accumulate over time steps. If these errors inflate the predicted rewards or value targets, the MSE loss amplifies the gradients, pushing value predictions further. This feedback loop can cause a value explosion, where the critic loss diverges (exceeding 10^{15} in our experiments), corrupting gradient signals and causing policy collapse.

The severity of this issue depends on the environment. In environments with small, bounded rewards (such as CartPole-v1, where each step yields a reward of $+1$), prediction errors are constrained, and standard MSE losses may suffice. In environments with larger, variable rewards (such as LunarLander-v3, where rewards span from approximately -100 to $+200$), the MSE amplification is pronounced, and training instability can emerge. Both environments are provided by the Gymnasium library (Towers et al., 2024).

This project investigates whether the symlog transformation, originally proposed as part of the DreamerV3 architecture, can be applied in isolation to address value explosion in a simplified imagination-based agent.

1.3 Aims and Objectives

Aim: To evaluate whether symlog-transformed losses improve training stability and prevent value divergence in imagination-based reinforcement learning compared to standard MSE losses.

Objectives:

1. **Implement a simplified imagination-based RL agent in PyTorch.** This agent operates in the raw state space using two-layer MLP networks with 256 hidden units for the actor, critic, and world model.

2. **Implement the symlog transformation and integrate it into training.** Symlog is applied to the targets of the critic and reward prediction losses, computing the MSE in symlog space.

3. **Compare three configurations:** (A) a model-free actor-critic baseline, (B) an imagination-augmented agent with standard MSE losses, and (C) an imagination-augmented agent with symlog-transformed MSE losses. These are evaluated on CartPole-v1 and LunarLander-v3 using five random seeds, yielding 30 runs.

4. **Analyze learning curves, loss trajectories, and world model prediction errors** to assess performance and stability.

1.4 Solution Approach

Our approach uses a simplified Dreamer-like architecture that retains imagination-based training but operates directly in the raw observation space. This design choice isolates the effects of symlog-transformed losses by avoiding the complexity of latent-space learning.

The world model consists of a transition model predicting next-state residuals and a reward model predicting immediate rewards. Both are implemented as two-layer MLPs with 256 hidden units. The actor and critic networks use the same architecture.

Training uses a Dyna-style scheme. At each step, the agent samples transitions from a replay buffer to update the world model. It then generates imagined transitions by rolling out the world model for a fixed horizon. These real and imagined transitions are used to update the actor and critic networks.

The configurations differ only in their loss target computations:

- In **Condition A** (model-free), the agent trains only on real transitions using standard MSE.
- In **Condition B** (imagination, standard MSE), the agent trains on real and imagined transitions using standard MSE.
- In **Condition C** (imagination, symlog MSE), the critic and reward prediction targets are symlog-transformed before computing the MSE loss.

All other hyperparameters are held constant. Each configuration is evaluated over five independent random seeds to account for stochastic variability.

1.5 Summary

This chapter outlined the context and motivation for studying symlog-transformed losses in imagination-based reinforcement learning. We described the sensitivity of standard MSE losses to reward scale, defined the project aims and objectives, and outlined our simplified architecture. Chapter 2 reviews the literature on these topics.

Chapter 2

Literature Review

This chapter reviews the literature relevant to the design and motivation of this project. It establishes the foundations of reinforcement learning, discusses model-based approaches, and traces the development of world models leading to DreamerV3 (Hafner et al., 2023). It also details the symlog transformation and the specific motivation for this work.

2.1 Reinforcement Learning Foundations

Reinforcement learning (RL) is a computational framework in which an agent learns to make sequential decisions by interacting with an environment to maximize a cumulative reward signal. The interaction is formalised as a Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition function, $R(s, a)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. At each time step t, the agent observes a state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$ according to its policy $\pi(a \mid s)$, receives a scalar reward $r_t = R(s_t, a_t)$, and transitions to a successor state $s_{t+1} \sim P(\cdot \mid s_t, a_t)$. The agent's objective is to find a policy π^* that maximises the expected discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k}. \tag{2.1}$$

Central to RL algorithms are value functions. The state-value function $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ measures the expected return from state s under policy π. Temporal-difference (TD) learning updates value estimates at each step using the observed reward and the estimated value of the successor state. The standard TD(0) update rule is:

$$V(s_t) \leftarrow V(s_t) + \alpha\big[r_t + \gamma V(s_{t+1}) - V(s_t)\big], \tag{2.2}$$

where α is the learning rate and the quantity in brackets is the TD error δ_t.

In actor-critic architectures, the agent maintains a parameterized policy π_θ (the actor) and a learned value function $V_\phi(s)$ (the critic) that serves as a baseline to reduce gradient variance. The actor parameters are updated using policy gradients:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t\Big], \tag{2.3}$$

where $\hat{A}_t$ is an estimator of the advantage function $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$.

2.2 Model-Based Reinforcement Learning and World Models

Model-free RL methods learn policies or value functions directly from observed transitions. Although successful, they typically require large numbers of environment interactions, making them sample-inefficient. Model-based RL addresses this by learning a predictive model of the environment (approximating the transition function $P(s' \mid s, a)$ and the reward function $R(s, a)$) and using it to generate simulated experience.

The Dyna architecture combines real-world acting, model learning, and planning. In Dyna, the learned world model generates simulated transitions that are used to update the policy or value function in the same manner as real experience. By interleaving planning steps with real interactions, the agent improves sample efficiency.

However, model-based planning introduces the challenge of compounding model error. When the learned model is imperfect, errors in predicted transitions accumulate over multi-step rollouts, causing simulated trajectories to diverge from reality. Polices optimized on these inaccurate simulations can perform poorly in the real environment.

To address compounding errors, modern algorithms learn world models and perform planning or "imagination" within a learned state space. The Dreamer family of algorithms has developed this approach for general-purpose applications. The third iteration of this family, DreamerV3 (Hafner et al., 2023), introduces scale-invariant learning techniques that enable a single set of hyperparameters to solve diverse environments.

2.3 The Symlog Transformation

A major challenge in general-purpose RL is the wide variation in reward and value scales. Environments with large reward scales produce large value targets, which generate large gradients in the critic's loss function, destabilising training.

The symmetric logarithm (symlog) transformation, introduced in DreamerV3 (Hafner et al., 2023), compresses both positive and negative values logarithmically while preserving the sign:

$$\text{symlog}(x) = \text{sign}(x) \cdot \ln\big(|x| + 1\big). \tag{2.4}$$

Its inverse, the symexp function, recovers the original scale:

$$\text{symexp}(x) = \text{sign}(x) \cdot \big(e^{|x|} - 1\big). \tag{2.5}$$

The symlog function is smooth, monotonic, and behaves approximately linearly near zero, transitioning to logarithmic compression for large magnitudes. This allows small rewards to retain resolution while large rewards are compressed, protecting gradient stability.

In DreamerV3 (Hafner et al., 2023), symlog is applied to the world model's reward predictor and to the critic's target return predictions. This keeps gradients stable across different reward scales.

2.4 Identified Gap and Project Motivation

DreamerV3 (Hafner et al., 2023) integrates multiple components, including a recurrent state-space model (RSSM), discrete latent representations, KL balancing, unimix distributions, symlog predictions, two-hot encoding, and return normalization. Because these components are evaluated together in complex environments, the specific contribution of the symlog transformation remains entangled with other scaling and planning mechanisms.

This project isolates the effect of the symlog transformation in a simplified setting. By using a feedforward world model in the raw state space, we examine how symlog-transformed losses affect training stability and prevent value divergence across environments with contrasting reward scales.

2.5 Summary

This chapter reviewed reinforcement learning, model-based planning, and the scale-invariance provided by the symlog transformation in DreamerV3 (Hafner et al., 2023). We identified the need to isolate the effects of symlog in a simplified architecture. Chapter 3 details the experimental design and methodology.

Chapter 3

Methodology

This chapter presents the methodology adopted to investigate the research objectives stated in Chapter 1. It describes the experimental design, the agent architectures under comparison, the symlog transformation and its integration into the learning pipeline, the environments used for evaluation, and the implementation details necessary for reproducibility. The chapter concludes with a summary that links the methodology to the results presented in Chapter 4.

3.1 Experimental Design

The experiment isolates the contributions of two choices: (i) a learned world model for imagination-based training, and (ii) the application of the symlog transformation to critic and reward prediction targets. We define three configurations, each tested across two environments with five random seeds, yielding 30 runs. Table 3.1 summarises the three conditions.

Table 3.1: Summary of the three experimental conditions

Condition	Agent	World Model	Imagination	Symlog
A	Model-Free AC	No	No	No
B	Imagination AC	Yes	Yes	No
C	Imagination AC + Symlog	Yes	Yes	Yes

Condition A is the baseline model-free actor-critic agent. Condition B adds a learned world model and uses imagined rollouts in a Dyna-style training loop. Condition C retains this imagination pipeline and applies the symlog transformation to the critic value and world model reward predictions. Comparing A to B assesses the effect of imagination-based training; comparing B to C evaluates the effect of symlog.

We evaluate each configuration on CartPole-v1 and LunarLander-v3. We use five random seeds per configuration to account for variation in initialization and sampling. Performance is measured as the mean episodic return averaged over evaluation episodes.

3.2 Agent Architecture

All agents share a common neural network foundation built upon two-layer multi-layer perceptrons (MLPs) with 256 hidden units per layer and ReLU activations. A shared `build_mlp` utility function constructs these networks, ensuring architectural consistency across all com-

ponents and conditions. The three core components (actor, critic, and world model) are described below.

3.2.1 Neural Network Components

Table 3.2 summarises the input, output, and training objective of each neural network component.

Table 3.2: Neural network components used across all conditions

Component	Input	Output	Loss	Conditions
Actor	State s	Categorical $\pi(\cdot\|s)$	Policy gradient	A, B, C
Critic	State s	Scalar $V(s)$	MSE on TD target	A, B, C
Dynamics head	(s, a)	State residual Δs	MSE	B, C
Reward head	(s, a)	Scalar $\hat{r}$	MSE	B, C

The **actor** network maps a state vector to a categorical probability distribution over the action space. During training, actions are sampled from this distribution; during evaluation, the action with the highest probability is selected. The actor is trained via policy gradient with an advantage baseline provided by the critic.

The **critic** network maps a state vector to a scalar estimate of the state value $V(s)$. Under the standard configurations (A and B), the critic outputs are in the original reward scale. Under the symlog configuration (Condition C), the critic outputs values in symlog space and the `get_value()` method applies the symexp inverse to recover values in the original scale for advantage computation.

The **world model** contains two MLP heads that share no parameters. The dynamics head takes a state-action pair and predicts a state residual Δs, such that the predicted next state is $\hat{s}' = s + \Delta s$. This residual formulation models only the change from the current state. The reward head takes the same state-action pair and predicts the scalar reward. When symlog is enabled (Condition C), the reward head outputs predictions in symlog space and the training target is transformed accordingly.

3.2.2 Model-Free Actor-Critic (Condition A)

Condition A implements a standard advantage actor-critic agent without a world model. At each training step, a mini-batch of transitions (s, a, r, s', d) is sampled uniformly from the replay buffer. The one-step temporal-difference (TD) target is computed as:

$$y = r + \gamma(1 - d)\, V(s'), \tag{3.1}$$

where γ is the discount factor and $d \in \{0, 1\}$ indicates whether the episode has terminated. The advantage is then:

$$A(s, a) = y - V(s). \tag{3.2}$$

The critic is updated by minimising the mean squared error between its prediction and the TD target:

$$\mathcal{L}_{\text{critic}} = \frac{1}{N} \sum_{i=1}^{N} \big(V(s_i) - y_i\big)^2. \tag{3.3}$$

The actor is updated using the policy gradient objective with an entropy regularisation term to encourage exploration:

$$\mathcal{L}_{\text{actor}} = -\frac{1}{N}\sum_{i=1}^{N}\left[\log \pi(a_i|s_i)\, A_i + \beta\, H(\pi(\cdot|s_i))\right], \tag{3.4}$$

where $H(\pi)$ denotes the entropy of the policy distribution and $\beta = 0.01$ is the entropy coefficient. The advantages A_i are normalised to zero mean and unit variance within each mini-batch to stabilise training.

3.2.3 Imagination-Based Actor-Critic (Conditions B and C)

Conditions B and C extend the model-free baseline with a learned world model and a Dyna-style training loop that supplements real experience with imagined rollouts. The training procedure comprises two phases that execute at each training step, plus an imagination phase that activates periodically.

Phase 1A: World Model Training. A mini-batch of real transitions is sampled from the replay buffer. The dynamics head is trained to predict the state residual $\Delta s = s' - s$ via MSE loss, and the reward head is trained to predict the observed reward. Two gradient steps are performed per training call to accelerate world model convergence.

Phase 1B: Actor-Critic Training on Real Data. Using the same mini-batch, the actor and critic are updated identically to the model-free procedure described in Section 3.2.2. This ensures that the agent continues to learn directly from real transitions regardless of the quality of the world model.

Phase 2: Imagination Rollouts. Every 10th training step, after a warmup period of 1,000 environment steps, the agent generates imagined experience. A batch of start states is sampled from the replay buffer, and the world model is used to simulate $H = 5$ step rollouts from each start state. At each imagined step, the actor selects an action from its current policy, and the world model predicts the next state and reward. The imagined trajectories are then used to compute n-step returns with critic bootstrapping at the final step:

$$G_t = \sum_{k=0}^{H-1} \gamma^k \hat{r}_{t+k} + \gamma^H V(\hat{s}_{t+H}), \tag{3.5}$$

where $\hat{r}$ and $\hat{s}$ denote the world model predictions. The actor and critic are then trained on this imagined data using the same loss functions as in Phase 1B, with advantage normalisation applied independently to the imagined batch.

This approach operates directly in the raw observation space rather than in a learned latent space, making it suitable for low-dimensional state spaces.

Algorithm 1 presents the pseudocode for the complete training loop of the imagination-based actor-critic.

3.3 Symlog Integration

The symlog transformation, introduced by Hafner et al. (2023), is a symmetric logarithmic function designed to compress the scale of regression targets while preserving their sign. It

Algorithm 1 Imagination-based actor-critic training loop

Input: Environment $\mathcal{E}$, replay buffer $\mathcal{B}$, networks π_θ, V_ϕ, f_ψ (world model)
Output: Trained policy π_θ

Initialise $\mathcal{B}$, π_θ, V_ϕ, f_ψ with random parameters
$s \leftarrow \mathcal{E}.\text{reset}()$
for $t = 1$ to T **do**
  $a \sim \pi_\theta(\cdot|s)$ ▷ Sample action from policy
  $s', r, d \leftarrow \mathcal{E}.\text{step}(a)$
  Store (s, a, r, s', d) in $\mathcal{B}$
  if $|\mathcal{B}| \geq$ batch_size **then**
    Sample mini-batch $\{(s_i, a_i, r_i, s'_i, d_i)\}$ from $\mathcal{B}$
    // Phase 1A: World model update (2 gradient steps)
    Update f_ψ on dynamics loss $\|\Delta\hat{s}_i - \Delta s_i\|^2$ and reward loss $\|\hat{r}_i - r_i\|^2$
    // Phase 1B: AC update on real data
    Compute TD targets $y_i = r_i + \gamma(1 - d_i)V_\phi(s'_i)$
    Update V_ϕ via Eq. (3.3), π_θ via Eq. (3.4)
    // Phase 2: Imagination (every 10 steps, after warmup)
    if $t \mod 10 = 0$ **and** $t > 1000$ **then**
      Sample start states $\{s_0^{(j)}\}$ from $\mathcal{B}$
      for $k = 0$ to $H - 1$ **do**
        $a_k^{(j)} \sim \pi_\theta(\cdot|s_k^{(j)})$
        $\hat{s}_{k+1}^{(j)}, \hat{r}_k^{(j)} \leftarrow f_\psi(s_k^{(j)}, a_k^{(j)})$
      end for
      Compute n-step returns $G^{(j)}$ via Eq. (3.5)
      Update V_ϕ, π_θ on imagined data
    end if
  end if
  $s \leftarrow s'$ (reset if d)
end for

addresses a common challenge in reinforcement learning: the reward scale varies dramatically across environments and over the course of training, which can destabilise gradient-based optimisation when raw values are used as regression targets.

3.3.1 Mathematical Definition

The symlog transformation and its inverse (symexp) are defined as:

$$\text{symlog}(x) = \text{sign}(x) \cdot \ln(|x| + 1), \tag{3.6}$$

$$\text{symexp}(x) = \text{sign}(x) \cdot \left(e^{|x|} - 1\right). \tag{3.7}$$

These functions satisfy $\text{symexp}(\text{symlog}(x)) = x$ for all x. The symlog function behaves approximately linearly near zero and logarithmically for large magnitudes, providing a smooth compression that is well-suited to targets whose scale is not known in advance.

3.3.2 Application Points

In Condition C, the symlog transformation is applied at two points in the learning pipeline. Table 3.3 contrasts the standard and symlog configurations.

Table 3.3: Application of the symlog transformation across conditions

Component	Standard (A, B)	Symlog (C)
Critic output space	Original scale	Symlog space
Critic TD target	$y = r + \gamma V(s')$	$\text{symlog}(r + \gamma\, \text{symexp}(V_{\text{raw}}(s')))$
Critic `get_value()`	$V(s)$	$\text{symexp}(V_{\text{raw}}(s))$
WM reward target	r	$\text{symlog}(r)$
WM reward prediction	$\hat{r}$	$\hat{r}_{\text{raw}}$ (in symlog space)
WM reward at inference	$\hat{r}$	$\text{symexp}(\hat{r}_{\text{raw}})$

When symlog is enabled, the critic network learns to predict values in the compressed symlog space. The MSE loss is computed between the raw network output and the symlog-transformed TD target. At inference time, the `get_value()` method applies symexp to recover the value estimate in the original reward scale, which is then used for advantage computation. Similarly, the world model reward head predicts rewards in symlog space; during imagination rollouts, symexp is applied to convert predicted rewards back to the original scale before computing n-step returns.

3.3.3 Numerical Stability

We use two measures to ensure numerical stability when using the symlog and symexp transformations. First, the input to symexp is clamped to the range $[-20, 20]$ to prevent overflow in the exponential computation. Second, gradient norms are clipped to a maximum of 1.0 for all networks across all conditions. This gradient clipping serves as a safeguard against training instability from large value predictions, particularly during early training when the world model is still inaccurate.

3.4 Environment Details

We evaluate the agents on two Gymnasium environments that differ in dimensionality, action space size, and reward structure: CartPole-v1 and LunarLander-v3.

3.4.1 CartPole-v1

CartPole-v1 is a classic control task in which the agent must balance a pole on a cart by applying left or right forces. The observation space is a four-dimensional continuous vector comprising cart position, cart velocity, pole angle, and pole angular velocity. The action space is discrete with two actions (push left, push right). A reward of $+1$ is received at every time step the pole remains upright, and the episode terminates when the pole angle exceeds $\pm 12°$, the cart moves beyond ± 2.4 units from the centre, or 500 steps are reached. The maximum achievable return is therefore 500. CartPole-v1 serves as a simple baseline environment with a narrow, non-negative reward range, where symlog compression is expected to have minimal impact.

3.4.2 LunarLander-v3

LunarLander-v3 is a more challenging control task in which the agent must land a spacecraft on a designated pad. The observation space is an eight-dimensional continuous vector encoding position, velocity, angle, angular velocity, and leg contact indicators. The action space is

discrete with four actions (do nothing, fire left engine, fire main engine, fire right engine). The reward structure is complex: the agent receives positive reward for moving toward the landing pad and for leg contact, but incurs penalties for fuel usage and crashes. Typical episodic returns range from approximately -200 (crash) to $+300$ (successful landing). LunarLander-v3 is included because its wide, mixed-sign reward range creates the conditions under which symlog compression is expected to be most beneficial.

3.4.3 Environment Selection Rationale

CartPole-v1 provides a low-dimensional, narrow-reward setting to verify basic functionality. LunarLander-v3 features higher dimensionality, a larger action space, and a variable reward signal to evaluate the benefits of imagination-based training and symlog scaling.

3.5 Implementation Details

3.5.1 Software Framework

All agents are implemented in Python using PyTorch for neural network construction, automatic differentiation, and optimisation. The environments are instantiated via the Gymnasium API. Training was conducted on CPU, as the network architectures and environments are computationally lightweight.

3.5.2 Hyperparameters

Table 3.4 lists the hyperparameters shared across all conditions. Where a hyperparameter applies only to conditions with a world model (B and C), this is indicated in the table.

Table 3.4: Hyperparameters used across all experimental conditions

Hyperparameter	Value	Conditions
Learning rate (all networks)	3×10^{-4}	A, B, C
Discount factor γ	0.99	A, B, C
Batch size	64	A, B, C
Replay buffer capacity	100,000	A, B, C
Entropy coefficient β	0.01	A, B, C
Gradient clip norm	1.0	A, B, C
Hidden layer size	256	A, B, C
Number of hidden layers	2	A, B, C
Activation function	ReLU	A, B, C
Imagination horizon H	5	B, C
Warmup steps before imagination	1,000	B, C
Imagination frequency	Every 10 steps	B, C
WM gradient steps per update	2	B, C
Symexp clamp range	$[-20, 20]$	C
Training steps (CartPole-v1)	100,000	A, B, C
Training steps (LunarLander-v3)	300,000	A, B, C

All networks use the Adam optimiser with the same learning rate. No learning rate scheduling or decay is applied. The hyperparameters were selected based on defaults from

the reinforcement learning literature and kept constant across all conditions to ensure a fair comparison.

3.5.3 Replay Buffer

A fixed-capacity replay buffer stores transitions as pre-allocated NumPy arrays. When the buffer is full, new transitions overwrite the oldest entries in a circular fashion. During training, mini-batches are sampled uniformly at random. The sampled arrays are converted to PyTorch tensors via zero-copy `torch.from_numpy()` calls, avoiding memory allocation and improving sampling throughput.

3.5.4 Evaluation Protocol

Agent performance is evaluated every 1,000 environment steps by running 10 evaluation episodes with the greedy policy (i.e., the action with the highest probability under the actor is selected deterministically). The mean and standard deviation of the episodic returns across these 10 episodes are recorded. This protocol is identical across all configurations. At reporting time, results are aggregated across the five seeds per configuration.

3.6 Summary

This chapter described the methodology for evaluating imagination-based training and symlog-transformed targets. We defined three configurations (A, B, and C) sharing the same network architecture, hyperparameters, and evaluation protocol. Chapter 4 presents the experimental results.

Chapter 4

Results

This chapter presents the experimental results. We evaluate three configurations across two Gymnasium environments, with each repeated using five random seeds. The configurations under comparison are:

- **Condition A (Model-Free Actor-Critic)**: a baseline actor-critic agent training on real environment interactions.
- **Condition B (Imagination AC, standard MSE)**: an imagination-augmented actor-critic using standard MSE losses.
- **Condition C (Imagination AC, Symlog MSE)**: an imagination-augmented actor-critic using symlog-transformed losses for the critic and reward prediction.

We present learning curves, critic loss stability, and world model prediction accuracy for each environment. Shaded regions in learning-curve plots denote the standard deviation across the five seeds.

4.1 CartPole-v1 Results

CartPole-v1 is a simple balancing task with a constant $+1$ reward per time step and a maximum episode length of 500. All agents were trained for 100 000 environment steps.

4.1.1 Learning Curves

Figure 4.1 shows the mean episodic return over training for each condition. The Model-Free baseline (Condition A) rises steadily and plateaus at approximately 350 return. Condition B (Imagination, standard MSE) displays higher sample efficiency in the early phase of training, reaching a mean return of roughly 400 by 60 000 steps, but subsequently plateaus and does not improve further. Condition C (Imagination, Symlog MSE) starts comparatively slower than Condition B but continues to improve beyond the 60 000-step mark and ultimately achieves the highest final return of approximately 430.

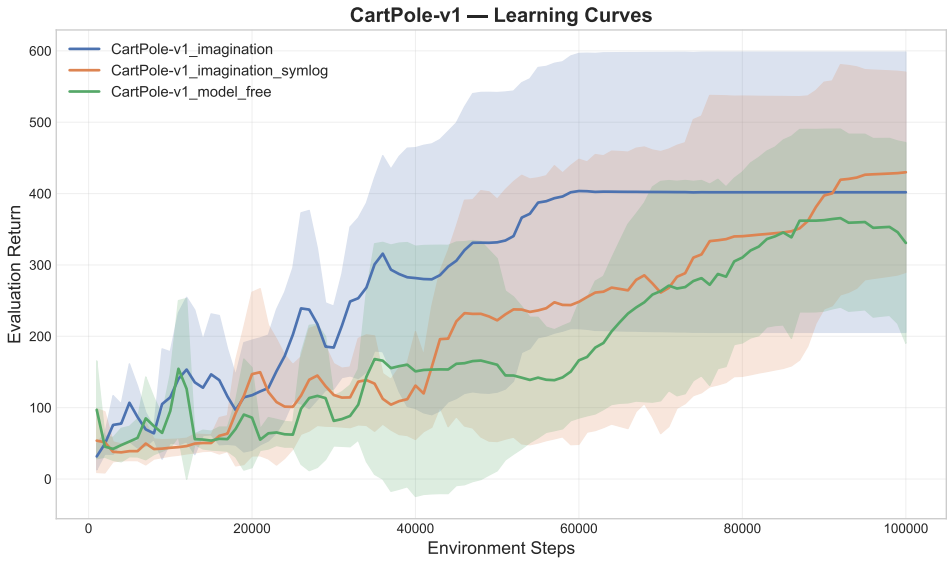

Figure 4.1: CartPole-v1 learning curves: mean episodic return (± 1 standard deviation) over 100 000 environment steps for each agent condition.

4.1.2 Critic Loss Stability

Figure 4.2 compares the critic loss trajectories of the two imagination-augmented conditions. Condition B (standard MSE) exhibits a pronounced loss explosion towards the end of training, with the critic loss escalating to approximately 5×10^{11}. In contrast, Condition C (Symlog MSE) maintains a near-zero critic loss throughout the entire training run, with no observable instability.

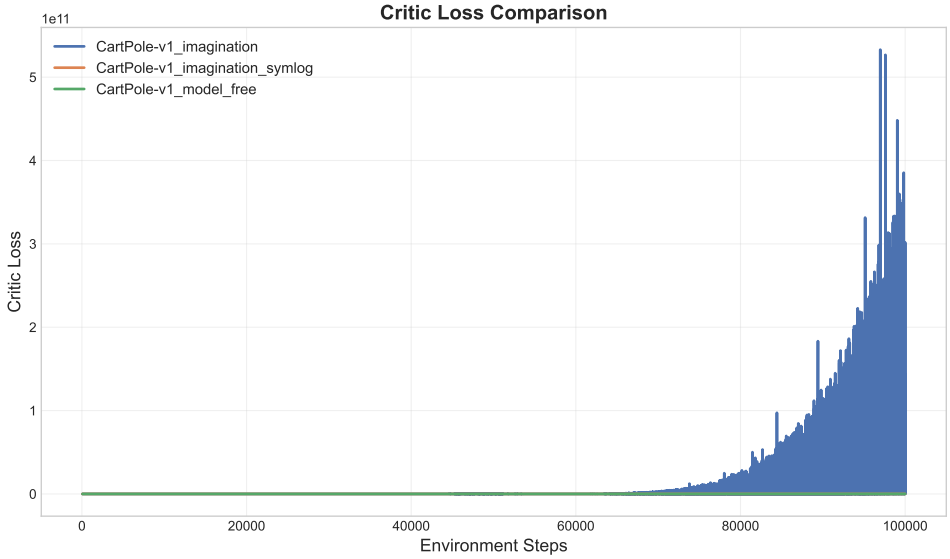

Figure 4.2: CartPole-v1 critic loss comparison between the two imagination-augmented conditions over 100 000 environment steps. Note the divergent y-axis scale caused by the loss explosion in Condition B.

4.1.3 World Model Accuracy

Figure 4.3 displays the world model prediction errors for both dynamics (next-state prediction) and reward prediction. Both imagination-augmented conditions learn the CartPole dynamics rapidly, with dynamics prediction error falling to near zero early in training. Reward prediction error is similarly negligible for both conditions, which is expected given that CartPole-v1 issues a constant $+1$ reward at every non-terminal step.

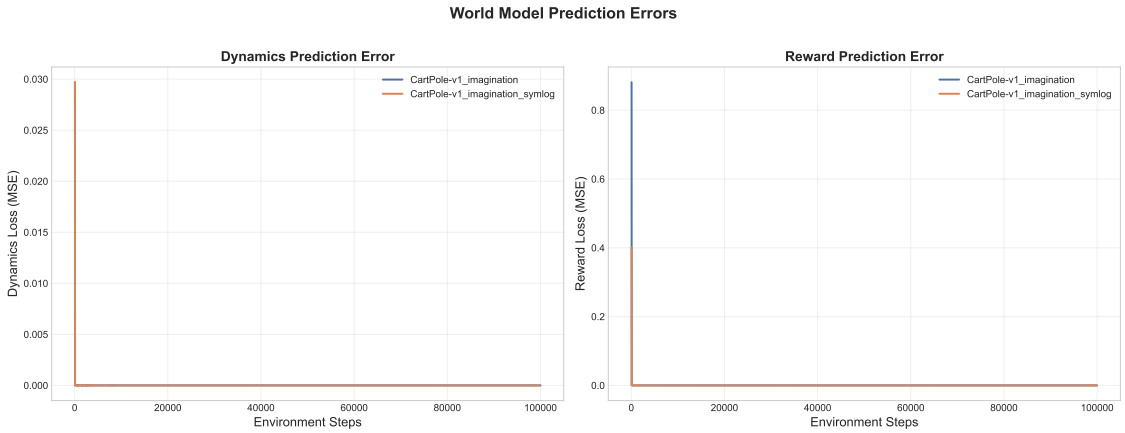

Figure 4.3: CartPole-v1 world model prediction error: dynamics MSE and reward MSE for Conditions B and C over 100 000 environment steps.

4.2 LunarLander-v3 Results

LunarLander-v3 is a continuous-control landing task with a shaped reward signal that spans a wide numerical range (from large negative penalties for crashing to positive bonuses for a controlled touchdown). All agents were trained for 300 000 environment steps.

4.2.1 Learning Curves

Figure 4.4 shows the mean episodic return for each condition. The Model-Free baseline (Condition A) converges to a stable mean return of approximately -140. Condition C (Imagination, Symlog MSE) converges to a comparable level of roughly -140, tracking the Model-Free agent closely. Condition B (Imagination, standard MSE) exhibits catastrophic failure: its mean return collapses to approximately -1000 early in training and never recovers, remaining at that floor for the duration of the 300 000-step budget.

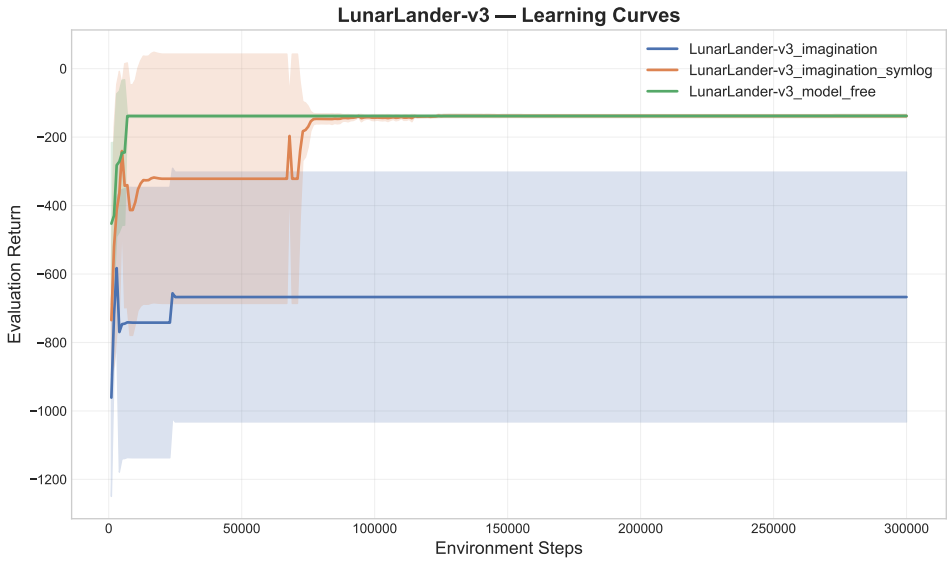

Figure 4.4: LunarLander-v3 learning curves: mean episodic return (± 1 standard deviation) over 300 000 environment steps for each agent condition.

4.2.2 Critic Loss Stability

Figure 4.5 compares the critic loss for the two imagination-augmented conditions in LunarLander-v3. Condition B (standard MSE) undergoes a critic loss explosion at approximately 100 000 steps, with the critic loss reaching the order of 10^{15}. Condition C (Symlog MSE) maintains a stable critic loss throughout the full 300 000 steps, with no runaway gradient dynamics.

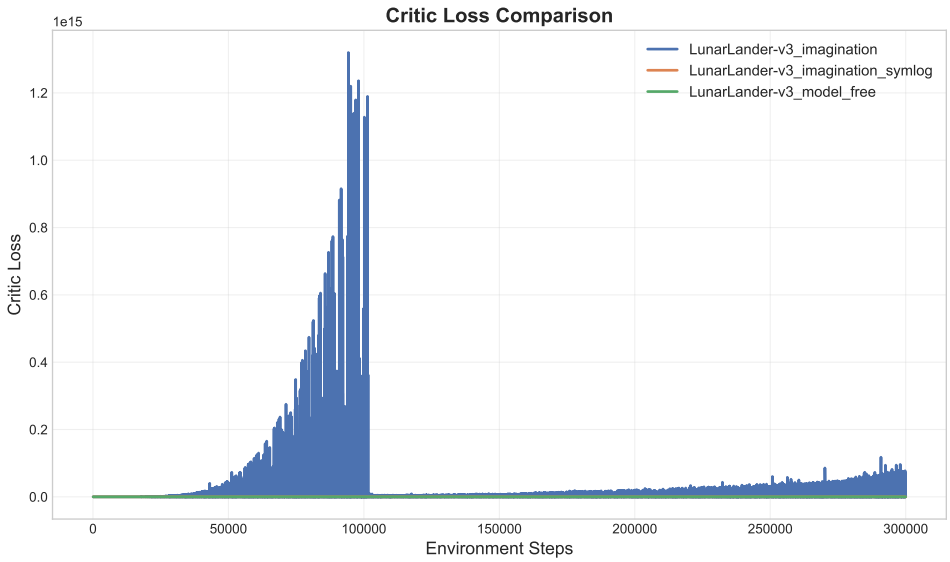

Figure 4.5: LunarLander-v3 critic loss comparison between the two imagination-augmented conditions over 300 000 environment steps. Condition B explodes to $\sim 10^{15}$; Condition C remains stable near zero.

4.2.3 World Model Accuracy

Figure 4.6 presents the world model prediction errors for LunarLander-v3. The dynamics prediction error is similar for both imagination-augmented conditions, indicating that both world models acquire a comparable representation of the environment's transition function. However, the reward prediction error differs: Condition B (standard MSE) exhibits large, fluctuating reward MSE, varying between 0 and approximately 450 MSE throughout training. Condition C (Symlog MSE) maintains a near-zero reward prediction error.

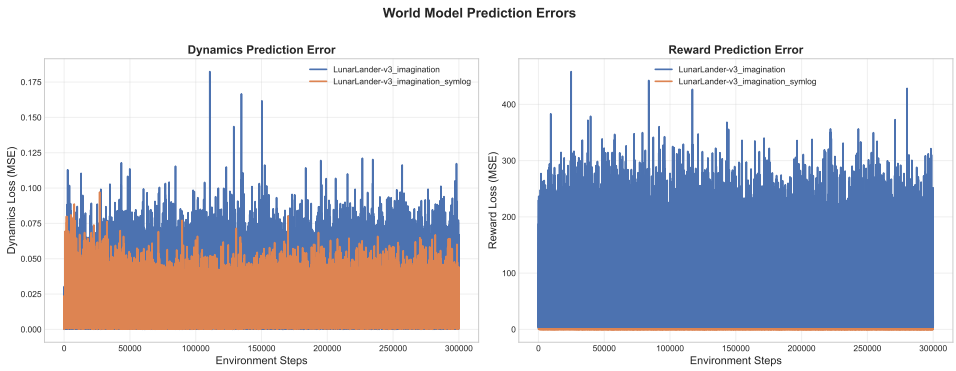

Figure 4.6: LunarLander-v3 world model prediction error: dynamics MSE and reward MSE for Conditions B and C over 300 000 environment steps.

4.3 Comparative Summary

Table 4.1 summarizes the quantitative and qualitative outcomes across the configurations and environments. For each pair, the table reports the approximate final mean episodic return, critic loss stability, and world model reward prediction error stability. The imagination-augmented agent benefits from symlog-transformed losses in both environments, particularly in LunarLander-v3 where the configuration without symlog collapses.

Table 4.1: Comparative summary of experimental results across conditions and environments.

Environment	Condition	Final Return	Critic Loss	WM Reward Error
CartPole-v1	A: Model-Free	~ 350	N/A	N/A
	B: Imagination (MSE)	~ 400	Exploded ($\sim 5 \times 10^{11}$)	Near zero
	C: Imagination (Symlog)	~ 430	Stable	Near zero
LunarLander-v3	A: Model-Free	~ -140	N/A	N/A
	B: Imagination (MSE)	~ -1000	Exploded ($\sim 10^{15}$)	Unstable (0 to 450)
	C: Imagination (Symlog)	~ -140	Stable	Near zero

4.4 Summary

This chapter presented the experimental results. In CartPole-v1, the imagination-augmented agent with symlog (Condition C) achieved the highest return and maintained stable critic losses, while the standard MSE variant (Condition B) showed late-stage critic loss divergence.

In LunarLander-v3, Condition B suffered critic loss divergence and policy collapse, whereas Condition C remained stable. Across both environments, dynamics prediction errors were similar, but reward prediction stability was higher in the symlog configuration. These results are discussed in Chapter 5.

Chapter 5

Discussion and Analysis

This chapter discusses the experimental results, compares them to existing literature, and outlines the limitations of the study. The findings demonstrate that target transformations are key to stabilizing training in imagination-based reinforcement learning.

5.1 Interpretation of Results

5.1.1 Effect of Imagination-Based Training

The comparison between Condition A (model-free actor-critic) and Condition B (imagination-based actor-critic with MSE losses) illustrates the trade-offs of learning from simulated experience. In CartPole-v1, Condition B showed higher sample efficiency in early training than Condition A, reaching near-optimal returns in fewer environment interactions (see fig. 4.1). This outcome is consistent with model-based reinforcement learning theory: generating simulated rollouts from a trained world model allows the agent to reuse transitions, reducing the sample complexity of learning.

However, model-based training introduces prediction error from the world model into the optimization process. When training on simulated trajectories, errors in the world model are propagated to the actor and critic updates. In CartPole-v1, which features a constant $+1$ reward and smooth state transitions, these compounding errors remain small. As shown in fig. 4.2, the critic loss for Condition B remained bounded.

In contrast, in LunarLander-v3, Condition B failed to learn and experienced policy collapse, with mean returns dropping to approximately -1000 (fig. 4.4). This indicates that imagination-based training without proper target scaling can destabilize policy learning.

5.1.2 The Value Explosion Problem

The failure of Condition B in LunarLander-v3 is associated with value function divergence. As shown in fig. 4.5, the critic loss for Condition B escalated to approximately 10^{15}, which corrupted the value estimates and the policy gradient signal.

This divergence is driven by the compounding of prediction errors across imagined rollouts. During imagination, the agent generates trajectories of horizon H by autoregressively applying the world model. At step t, the predicted next state contains prediction errors. Because subsequent steps are conditioned on previous predictions, these errors accumulate. Over an H-step horizon, the cumulative prediction error grows.

In LunarLander-v3, this accumulation is amplified by the reward structure, which includes terminal rewards of -100 and $+100$ along with continuous shaping rewards. When the critic

fits these variable target values using standard MSE loss, prediction errors result in large squared residuals. For example, a difference of 200 between the target and predicted values yields a squared error of 40,000 before temporal compounding is factored in.

The resulting large losses produce large gradients, which push the critic's parameters to produce extreme predictions. This feedback loop (where prediction errors yield large gradients that increase subsequent prediction errors) leads to divergence. This instability motivated the development of symlog losses in DreamerV3 (Hafner et al., 2023).

5.1.3 Symlog as a Stabilizer

Condition C demonstrates that symlog-transformed losses prevent value divergence. In LunarLander-v3, the critic loss under Condition C remained stable throughout training (fig. 4.5), while the mean returns stabilized at approximately -140, matching the model-free baseline (fig. 4.4).

The symlog function, defined as:

$$\text{symlog}(x) = \text{sign}(x) \ln(|x| + 1) \tag{5.1}$$

compresses values logarithmically while preserving their sign. A target of -100 maps to $\text{symlog}(-100) \approx -4.62$, and $+100$ maps to $\approx +4.62$. The dynamic range is thus compressed.

This compression bounds the squared residuals. The maximum squared error in symlog space is restricted to approximately $(4.62 - (-4.62))^2 \approx 85.4$, compared to values up to 40,000 in the original space. Bounding the loss values constrains the gradient magnitudes, preventing the feedback loop that drives divergence.

Additionally, the world model's reward predictor benefits from the symlog transformation. As shown in fig. 4.6, the reward prediction error under Condition C is lower than that of Condition B. Predicting in the compressed symlog space simplifies regression because the target values are bounded.

5.2 Comparison with Existing Literature

These results support the findings of Hafner et al. (2023) regarding the role of symlog transformations in stabilizing world-model-based learning. While DreamerV3 combines symlog predictions with several other components (two-hot encoding, KL balancing, and the RSSM), our study isolates the effect of symlog. By using raw state-space observations, a feedforward MLP world model, and standard actor-critic updates, we verify that the symlog transformation alone prevents value divergence.

Our hybrid Dyna-style training approach, which uses both real and simulated transitions, connects to classic planning architectures. Interleaving model-free updates on real data with planning on simulated data stabilizes learning when the world model is still inaccurate early in training. Real transitions provide a baseline that helps preserve policy quality.

5.3 Limitations

Several limitations of this study should be noted:

- **Simplified world model architecture:** The feedforward MLP world model operating in the raw observation space is simpler than recurrent latent-space models like the RSSM in DreamerV3. This simplification isolates the effect of symlog but limits the capacity to capture long-term dependencies.

- **Discrete action spaces:** We evaluated the configurations on environments with discrete action spaces. The interaction between symlog transformations and continuous action spaces remains to be evaluated.
- **Limited environment diversity:** The study is restricted to CartPole-v1 and LunarLander-v3. Evaluating a wider range of environments with diverse reward scales would strengthen the findings.
- **Omission of two-hot encoding:** We did not evaluate two-hot categorical encoding, which DreamerV3 combines with symlog.
- **Incomplete task performance:** While Condition C prevented value explosion in LunarLander-v3, the final returns remained around -140, falling short of the solved threshold of $+200$. This indicates that while symlog stabilizes training, additional architectural or algorithmic components are needed for high performance.

5.4 Summary

This chapter analyzed the experimental results, focusing on error compounding and the stabilizing properties of the symlog transformation. The symlog function prevents value divergence by bounding the dynamic range of the regression targets. Our results isolate the contribution of symlog in a simplified setting. The limitations identified here suggest directions for future work, which are discussed in Chapter 6.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This project evaluated the effect of symlog-transformed losses on the stability and performance of imagination-based reinforcement learning agents. Inspired by DreamerV3 (Hafner et al., 2023), we isolated and evaluated the contribution of symlog in a simplified setting.

We implemented a simplified Dreamer-like agent operating in raw state space without the RSSM, two-hot encoding, or KL balancing. We compared three configurations: model-free actor-critic (Condition A), imagination-based actor-critic with standard MSE losses (Condition B), and imagination-based actor-critic with symlog-transformed losses (Condition C). These were evaluated on CartPole-v1 and LunarLander-v3.

The key findings are:

- **Imagination-based training improves sample efficiency in benign settings.** In CartPole-v1, Condition B reached near-optimal performance in fewer steps than Condition A, supporting the premise that learning from simulated experience can improve sample efficiency.
- **Standard MSE losses can cause value function divergence in imagination-based agents.** In LunarLander-v3, where rewards span from -100 to $+100$, Condition B failed to learn. The critic loss escalated to magnitudes exceeding 10^{15}, leading to policy collapse with mean returns of approximately -1000.
- **Symlog-transformed losses prevent value divergence and reward prediction instability.** Condition C maintained stable critic losses in both environments. The logarithmic compression of the symlog function bounds gradient magnitudes, preventing the feedback loop between prediction errors and large parameter updates.
- **Symlog provides stability independent of other architectural components.** The stabilizing effect of symlog was observed without other DreamerV3 components, indicating that symlog directly addresses value divergence in imagination-based learning.

This study provides an evaluation of the symlog transformation in isolation. By controlling for other components of DreamerV3, we show that symlog alone is sufficient to prevent value function divergence. This suggests that symlog-transformed losses are a useful design choice for agents training on simulated trajectories.

6.2 Future Work

The limitations identified in Chapter 5 suggest several directions for future research.

6.2.1 Short-Term Extensions

Continuous action spaces. A direct extension is to test the configurations in continuous action spaces, such as MountainCarContinuous-v0 or HalfCheetah-v4. Continuous control tasks feature different reward distributions and require policy gradient methods that may interact differently with the symlog transformation.

Two-hot encoding. DreamerV3 combines symlog with two-hot categorical encoding for value and reward predictions (Hafner et al., 2023). Future work could introduce two-hot encoding as a separate experimental configuration to analyze the individual and combined effects of symlog and two-hot encoding.

Reward scaling experiments. To characterize the sensitivity of imagination-based training to reward scale, experiments could vary reward scales by multiplicative factors (such as $100\times$ or $0.01\times$). This would show the thresholds at which value divergence occurs under standard MSE losses.

6.2.2 Long-Term Investigations

Recurrent latent dynamics. Replacing the MLP-based world model with a Recurrent State-Space Model (RSSM) would bring the architecture closer to DreamerV3. This would allow investigation of whether symlog remains necessary when using latent-space world models.

Complex environments. Evaluating the agents on pixel-based observations or tasks with longer time horizons, such as Atari games, would test the scalability of the approach.

Alternative normalization techniques. Future work could compare symlog with other normalization methods, such as PopArt (adaptive normalization of targets) or running-mean normalization, to establish performance trade-offs.

6.3 Summary

This chapter summarized the findings, highlighting the role of symlog-transformed losses in preventing value divergence during imagination-based reinforcement learning. The results show that symlog is an effective stabilization mechanism in isolation. Future work includes evaluating continuous action spaces, alternative encoding methods, and more complex environments.

Chapter 7

Reflection

This chapter offers a reflection on the project. We discuss the knowledge and skills we developed, the decisions and challenges we encountered, and how the project has shaped our perspective on reinforcement learning research.

Developed knowledge and skills. This project required us to work with reinforcement learning concepts in detail. We implemented actor-critic agents, world models, and imagination-based training loops in PyTorch, which required understanding each component at the level of tensor operations rather than relying on high-level libraries. Through this process, we gained experience with policy gradient methods, value function estimation, and the integration of model-free and model-based learning.

We also gained practical experience with numerical stability in deep learning. Concepts such as NaN propagation, gradient explosion, and the sensitivity of neural networks to target magnitudes were previously theoretical. Debugging the failures of Condition B, where critic losses diverged to 10^{15}, and gradients became undefined, provided experience with these issues. This taught us to consider the numerical properties of loss functions alongside their theoretical formulations.

In addition, we developed our skills in experimental design and scientific methodology. Designing a controlled comparison across three configurations, managing random seeds for reproducibility, and interpreting results conservatively are competencies that apply to research in general.

Decision-Making and Problem-Solving. Several decisions shaped the trajectory of the project. The most significant was the transition from pure imagination-based training to a hybrid Dyna-style approach. Our initial implementation trained the actor and critic exclusively on imagined trajectories generated by the world model. This approach did not succeed: the policy collapsed early in training, even in CartPole-v1. We identified that the world model's early inaccuracies produced trajectories that did not match the true dynamics, leading the policy to optimize for incorrect transitions.

The decision to interleave real and imagined experience by training on both actual transitions and world model rollouts resolved this issue. This approach reduced the sample efficiency advantages of pure imagination training but produced stable learning dynamics. This experience indicated that the accuracy of the world model limits the utility of imagination-based training.

Another problem-solving task arises from debugging NaN values during training. We traced the issue to the `symexp` function (the inverse of symlog), where large inputs to the exponential function produced floating-point overflow. To resolve this, we clamped the input

to `symexp` to a safe range and applied gradient clipping to the optimizers. These are common practices in deep reinforcement learning, but verifying their utility through implementation was instructive.

Finally, the decision to work in the raw state space rather than implementing pixel-based observations and a full RSSM was a choice to manage scope. We decided that a clear, interpretable comparison would be more valuable than a partial implementation of the full DreamerV3 architecture. The clarity of the results supports this decision.

Challenges Encountered. The primary challenge was achieving stable training for the imagination-based agent. Instability in imagination-based RL can be sudden. A training run can collapse within a few episodes if the value function diverges. This made debugging difficult, as the visible failure (such as a NaN in the policy logits) was often separated from the root cause (such as a compounding error in the world model's reward predictions several hundred updates earlier).

Managing the computational demands of the experiment was also a challenge. Running three configurations in two environments with multiple random seeds required managing parallel processes. We encountered CPU thrashing when launching too many runs simultaneously, which we addressed by implementing a sequential execution strategy.

Balancing scope with feasibility was a constant consideration. We initially planned to implement the full DreamerV3 architecture, including RSSM, two-hot encoding, and KL-balanced world model training. However, we recognize that each additional component would introduce confounding factors, making it harder to isolate the role of symlog. Simplifying the setup clarified the findings.

What We Would Do Differently. In hindsight, several aspects of our approach could have been improved. We would have started with CartPole-v1 exclusively and only moved to LunarLander-v3 after establishing stable training in the simpler environment. Instead, we attempted to develop both environments in parallel, which complicated debugging when bugs manifested differently across the two settings.

We would also have implemented unit tests for the symlog and symexp functions earlier in the development process. Debugging time could have been reduced if we had verified the numerical properties of these functions (such as verifying that $\text{symexp}(\text{symlog}(x)) = x$ and that the gradients remain finite) before integrating them into the training loop.

Finally, we would have allocated more time for hyperparameter tuning, particularly for the imagination horizon length and the ratio of real-to-imagined updates. These parameters affect training stability and sample efficiency, and our choices were based on values reported in the literature rather than systematic optimization for our specific architecture.

Impact on Future Development. This project has deepened our understanding of the relationship between numerical stability and algorithmic design in deep reinforcement learning. We learned that the choice of loss parameterization can determine whether an algorithm succeeds, regardless of its theoretical soundness.

The experience of building a research codebase from scratch, including modular agent components, experiment configuration, and results logging, has increased our confidence in contributing to future research. We now have a template for conducting controlled experiments in RL.

More broadly, the project has developed our interest in world model research. The idea that agents can plan within a learned world model is a promising direction that we look forward to exploring in future academic and professional work.

7.1 Summary

This reflection documented the learning journey accompanying the technical work. The project required developing technical skills in PyTorch implementation and numerical debugging, making scoping decisions under time constraints, and conducting experimental comparisons. The challenges encountered, particularly value function divergence and the instability of pure imagination training, were key learning points. We are confident that the insights gained will inform our future work in artificial intelligence.

References

Hafner, D., Pasukonis, J., Ba, J. and Lillicrap, T. (2023), Mastering diverse domains through world models, *in* 'Advances in Neural Information Processing Systems (NeurIPS)', Vol. 36.

Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., KG, A., Krimmel, M., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J. J., Tan, A. and Younis, O. G. (2024), 'Gymnasium: A standard interface for reinforcement learning environments', *arXiv preprint arXiv:2407.17032* .

Appendix A

Hyperparameter Configuration

This appendix provides a comprehensive reference of all hyperparameters used throughout the experiments presented in this report. Unless otherwise stated, all configurations were held constant across the three experimental conditions (model-free, imagination, and imagination with symlog) to ensure a fair comparison.

A.1 Agent and Training Hyperparameters

Table A.1 lists every hyperparameter governing the agent architecture, optimisation procedure, and training loop.

A.2 Per-Environment Step Budgets

Each environment was allocated a fixed total interaction budget, chosen to provide sufficient training time for convergence while remaining computationally tractable. Table A.2 summarises the step budgets used.

The step budget for `LunarLander-v3` is three times that of `CartPole-v1`, reflecting its higher-dimensional observation and action spaces and consequently slower convergence characteristics.

A.3 Symlog Transform Implementation

The symlog and symexp functions, adapted from Hafner et al. (2023), provide a bijective pair of transformations that compress the scale of target values while preserving their sign. Listing A.1 presents the PyTorch implementations used in this project.

```
def symlog(x):
    return torch.sign(x) * torch.log1p(torch.abs(x))

def symexp(x):
    return torch.sign(x) * (
        torch.exp(torch.clamp(torch.abs(x), max=20.0)) - 1
    )

def symlog_mse_loss(predictions, targets):
    return F.mse_loss(predictions, symlog(targets))
```

Table A.1: Complete hyperparameter configuration used across all experiments.

Hyperparameter	**Value**	**Description**
Optimisation		
Learning rate	3×10^{-4}	Adam optimiser, shared by all networks
Gradient clip norm	1.0	Maximum gradient norm for clipping
Batch size	64	Mini-batch size sampled from replay buffer
Reinforcement Learning		
Discount factor (γ)	0.99	Temporal discount applied to future rewards
Entropy coefficient	0.01	Actor entropy bonus coefficient
Replay Buffer		
Buffer size	100,000	Maximum replay buffer capacity
Network Architecture		
Hidden dimensions	(256, 256)	MLP hidden layer sizes (two layers)
World Model & Imagination		
Imagination horizon	5	Number of steps to imagine forward
Warmup steps	1,000	Random action steps before imagination begins
WM train steps	2	World model gradient steps per training call
Imagination train ratio	10	Imagination update performed every N steps
Evaluation		
Evaluation frequency	1,000	Evaluate every N environment steps
Evaluation episodes	10	Number of episodes per evaluation
System		
Device	auto	CUDA if available, otherwise CPU

Table A.2: Total environment interaction steps allocated per environment.

Environment	**Total Steps**
CartPole-v1	100,000
LunarLander-v3	300,000

Listing A.1: Implementation of the symlog and symexp transforms and the associated loss function.

The `symlog` function compresses large magnitudes via $\text{sign}(x) \cdot \ln(1+|x|)$, ensuring that the world model's reward and value predictions operate on a numerically stable scale. The inverse `symexp` function recovers the original scale, with an internal clamp at $|x| \leq 20$ to prevent numerical overflow during exponentiation. The composite loss function `symlog_mse_loss` applies the symlog transform to the regression targets before computing the mean squared error, as described in Chapter 3.